EOCS

Egyptian Olympiad in Computational Science

Final Round Problem Set

2026

Computational Fields

Biology • Chemistry • Physics • Mathematics

Contents

1. Computational Physics

Problem 1.1

Layla's Sprinkler Experiment

Many quantities in science—areas, integrals, expectation values, and the behavior of huge collections of particles—are far easier to check than to solve directly. Monte Carlo methods exploit this idea: instead of solving a problem analytically, scientists scatter random samples across the space of possibilities and use the fraction satisfying a condition to estimate the answer.

This works because of the law of large numbers: as the number of random samples N grows, an average computed over them converges to the true value, with a statistical error that generally shrinks as N increases. This makes Monte Carlo methods useful whenever a direct calculation is difficult, even for something as simple as estimating π.

Layla, a physics student, sets up a square garden bed measuring $2\,\text{m} \times 2\,\text{m}$, with a circular flowerbed of radius $1\,\text{m}$ at its center. If water droplets from a sprinkler land uniformly at random across the square, the fraction landing inside the circular flowerbed reflects the ratio of the two areas—a ratio directly related to π.

Help Layla investigate this using Python.

Subtask 1 (6 pts): The sprinkler experiment

Simulate $N = 100{,}000$ droplets landing at uniformly random positions across the square garden bed, and use the fraction landing inside the circular flowerbed to estimate π.

Output: Print Layla's estimated value of π and its absolute error relative to the true value. Create a scatter plot of the droplets, using different colors for droplets that land inside and outside the circular flowerbed.

Subtask 2 (7 pts): How much water does Layla need?

Repeat the experiment using $N = 100$, 1,000, 10,000, 100,000, and 1,000,000 droplets, and track the absolute error of each estimate.

Output: Print a table showing N and the estimation error for each case. Plot estimation error against N using regular axes, and print one sentence stating whether the error generally becomes smaller as N increases.

Subtask 3 (7 pts): Is Layla's sprinkler reliable?

Using $N = 10{,}000$ droplets, repeat the experiment independently 30 times.

Output: Print the mean and standard deviation of the 30 estimates. Create a histogram of the estimates. In one printed sentence, state whether the estimates remain consistently close to π or vary substantially between runs.

Problem 1.2

Two-Body Motion

The motion of planets and stars is governed by gravity. For a system containing only two bodies, the equations of motion can be solved analytically in ideal cases. Numerical methods, however, provide a simple way to calculate their positions step by step and form the basis for more complicated simulations involving many bodies.

Consider two bodies interacting only through their mutual gravitational force. Their motion is calculated numerically using small time steps.

Given

$$G = 1, \qquad m_1 = 8, \qquad m_2 = 1, \qquad \Delta t = 0.01, \qquad T = 4.$$

$$\vec{r}_1(0) = (-0.2, 0), \qquad \vec{r}_2(0) = (0.8, 0.1),$$

$$\vec{v}_1(0) = (0, 0.05), \qquad \vec{v}_2(0) = (-0.05, 2.1).$$

The gravitational force exerted by body 1 on body 2 is

$$\vec{F}_{12} = G\frac{m_1 m_2}{r^3}\vec{r},$$

Use the Euler method,

$$\vec{v}_i(t + \Delta t) = \vec{v}_i(t) + \vec{a}_i(t)\Delta t,$$

$$\vec{r}_i(t + \Delta t) = \vec{r}_i(t) + \vec{v}_i(t + \Delta t)\Delta t.$$

Simulate the system from $t = 0$ to $t = T$.

Tasks

(a) **(4 pts)** Calculate the initial relative position $\vec{r}_0$, separation r_0, and gravitational force $\vec{F}_{12}$. Calculate the initial accelerations

$$(a_{1x}, a_{1y}) \qquad \text{and} \qquad (a_{2x}, a_{2y}).$$

(b) **(5 pts)** Simulate the system from $t = 0$ to $t = 4$ using the given Euler method and report the final positions of both bodies:

$$(x_1, y_1) \qquad \text{and} \qquad (x_2, y_2).$$

(c) **(5 pts)** Plot both trajectories in the $x - y$ plane, including their initial and final positions.

(d) **(6 pts)** Calculate the separation

$$r(t) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

throughout the simulation and report

$$r_{\min} \qquad \text{and} \qquad r_{\max}.$$

Problem 1.3

Mariam's Pendulum Experiments

Mariam is investigating oscillatory systems in the EOCS laboratory. She starts with a double pendulum and gradually modifies the experimental setup to investigate the behavior of large-angle oscillations and coupled pendulums.

Unless otherwise stated, all simulations in this problem must be performed using the **Velocity Verlet integrator**.

For every simulation, use

$$\Delta t = 0.01 \text{ s}.$$

When numerical quantities are requested, use the values obtained from your simulation rather than analytical approximations.

Subtask 1 (4 pts): The double pendulum

Mariam first builds a double pendulum. The system consists of two point masses connected by two massless, rigid rods. The first rod has length L_1 and connects the fixed support to the first mass, while the second rod has length L_2 and connects the first mass to the second mass.

The angles θ_1 and θ_2 are measured from the downward vertical direction. Thus, $\theta_i = 0$ corresponds to the corresponding rod pointing directly downward.

Use

$$m_1 = m_2 = 1.0 \text{ kg}, \qquad L_1 = L_2 = 1.0 \text{ m}, \qquad g = 9.81 \text{ m s}^{-2}.$$

The equations of motion of the double pendulum are

$$\ddot{\theta}_1 = \frac{-g(2m_1 + m_2)\sin\theta_1 - m_2 g \sin(\theta_1 - 2\theta_2) - 2m_2 \sin(\theta_1 - \theta_2)\left[L_2\dot{\theta}_2^2 + L_1\dot{\theta}_1^2\cos(\theta_1 - \theta_2)\right]}{L_1\left[2m_1 + m_2 - m_2\cos(2\theta_1 - 2\theta_2)\right]}, \tag{1}$$

$$\ddot{\theta}_2 = \frac{2\sin(\theta_1 - \theta_2)\left[L_1\dot{\theta}_1^2(m_1 + m_2) + g(m_1 + m_2)\cos\theta_1 + L_2\dot{\theta}_2^2 m_2\cos(\theta_1 - \theta_2)\right]}{L_2\left[2m_1 + m_2 - m_2\cos(2\theta_1 - 2\theta_2)\right]}. \tag{2}$$

Mariam releases the pendulum from rest with

$$\theta_1(0) = 30^\circ, \qquad \theta_2(0) = 0^\circ, \qquad \dot{\theta}_1(0) = \dot{\theta}_2(0) = 0.$$

Simulate the system for

$$0 \le t \le 20 \text{ s}.$$

Output:

(a) Plot $\theta_1(t)$ and $\theta_2(t)$ against time on the same graph.

(b) Plot the trajectory of the second mass in the Cartesian x–y plane.

(c) Calculate the total mechanical energy of the double pendulum at every timestep and plot its relative error with respect to its initial value:

$$\frac{|E(t) - E(0)|}{|E(0)|}.$$

(d) Report the maximum relative energy error during the simulation.

Subtask 2 (5 pts): The large-angle pendulum

After finishing the double-pendulum experiment, Mariam decides to return to a simpler system. She builds a **single pendulum**, consisting of one point mass attached to a massless, rigid rod.

Unlike the small-angle approximation commonly used for simple harmonic motion, Mariam wants to investigate the motion when the pendulum starts at a large angular displacement.

The pendulum has mass m and length L. Its angular displacement θ is measured from the downward vertical direction.

Use

$$m = 1.0 \text{ kg}, \qquad L = 1.0 \text{ m}, \qquad g = 9.81 \text{ m s}^{-2}.$$

The angular acceleration of the undamped pendulum is

$$\ddot{\theta} = -\frac{g}{L}\sin\theta.$$

Mariam then places the pendulum inside a chamber where air resistance causes its oscillations to gradually decrease. She models the effect of air resistance by adding the damping term

$$-k\dot{\theta}$$

to the angular equation of motion, where

$$k = 0.30 \text{ s}^{-1}.$$

Therefore, the simulation must account for both gravity and the specified air resistance.

Mariam releases the **single pendulum** from rest with the large initial displacement

$$\theta(0) = 120^\circ, \qquad \dot{\theta}(0) = 0.$$

Simulate the pendulum for

$$0 \le t \le 30 \text{ s}.$$

Output:

(a) Plot $\theta(t)$ against time.

(b) Plot $\dot{\theta}(t)$ against time.

(c) Plot the phase-space trajectory $(\theta, \dot{\theta})$.

(d) Determine numerically the first time at which the absolute angular displacement becomes smaller than

$$|\theta| < 1^\circ$$

and remains below 1° for the rest of the simulation.

Subtask 3 (5 pts): Mariam couples two pendulums

Mariam now changes her apparatus once again. She removes the second rod from the previous experiment and constructs **two separate single pendulums**.

The two pendulums are identical. Their suspension points are fixed at different horizontal positions and are separated by a distance d.

Each pendulum consists of a point mass m attached to a massless, rigid rod of length L. The two rods are free to swing in the same vertical plane.

The suspension points are

$$P_1 = (0, 0), \qquad P_2 = (d, 0).$$

The angular displacement of each pendulum is measured from the downward vertical direction. Therefore, the positions of the two masses are

$$\mathbf{r}_1 = \begin{pmatrix} L \sin\theta_1 \\ -L \cos\theta_1 \end{pmatrix},$$

and

$$\mathbf{r}_2 = \begin{pmatrix} d + L \sin\theta_2 \\ -L \cos\theta_2 \end{pmatrix}.$$

The two masses are connected by a light spring. The spring is attached directly between the two masses and has spring constant K and natural length L_0.

The instantaneous length of the spring is

$$\ell(t) = \|\mathbf{r}_2(t) - \mathbf{r}_1(t)\|.$$

The spring force has magnitude

$$F_s = K\,(\ell - L_0)\,.$$

The spring force acts along the line joining the two masses. The direction of this force must be taken into account when determining its effect on the motion of each pendulum.

Use

$$m = 1.0 \text{ kg}, \qquad L = 1.0 \text{ m}, \qquad d = 2.0 \text{ m},$$

$$K = 4.0\ \mathrm{N\,m^{-1}}, \qquad L_0 = 2.0\ \mathrm{m}, \qquad g = 9.81\ \mathrm{m\,s^{-2}}.$$

Initially, the spring is at its natural length when both pendulums are vertical.

Mariam releases the two pendulums from rest with

$$\theta_1(0) = 20^\circ, \qquad \theta_2(0) = -20^\circ,$$

$$\dot{\theta}_1(0) = \dot{\theta}_2(0) = 0.$$

There is no air resistance in this experiment.

Simulate the system for

$$0 \le t \le 30\ \mathrm{s}.$$

Output:

(a) Plot $\theta_1(t)$ and $\theta_2(t)$ against time on the same graph.

(b) Plot the instantaneous spring length $\ell(t)$ against time.

(c) Plot the horizontal positions of the two masses against time.

(d) Calculate and plot the total mechanical energy of the system against time.

Subtask 4 (6 pts): Mariam observes the coupled system

Mariam now wants to understand how the spring affects the relationship between the motions of the two pendulums.

Using the exact same physical parameters and initial conditions from Subtask 3, simulate the coupled system for

$$0 \le t \le 40\ \mathrm{s}.$$

Use the same Velocity Verlet integrator and timestep

$$\Delta t = 0.01\ \mathrm{s}.$$

Construct the phase-space trajectories for both pendulums.

Output:

(a) Plot

$$\theta_1(t) \quad \text{against} \quad \dot{\theta}_1(t)$$

as the phase-space trajectory of the first pendulum.

(b) Plot

$$\theta_2(t) \quad \text{against} \quad \dot{\theta}_2(t)$$

as the phase-space trajectory of the second pendulum.

(c) Plot both phase-space trajectories on the same graph, using appropriate labels and a legend.

(d) From the numerical results, determine whether the two pendulums predominantly move in phase or out of phase. Support your conclusion using the simulated angular displacements.

Problem 1.4

Omar's Long-Distance Challenge

Omar is working with his school's experimental sports-physics team. The team has built a small launcher capable of throwing a ball at a controlled initial speed and launch angle.

At first, Omar assumes that the ball moves through a uniform atmosphere. However, after comparing his simulations with experimental measurements, he realizes that the air becomes less dense as the ball gains altitude.

The team then asks a more difficult question:

What launch angle allows the ball to travel the greatest horizontal distance when air resistance, atmospheric variation, and repeated bounces are all taken into account?

Omar decides to investigate the problem computationally.

General simulation requirements

Unless otherwise stated, all simulations in this problem must use the **Velocity Verlet integrator**.

Use a fixed timestep of

$$\Delta t = 0.001 \text{ s}.$$

The motion takes place in a two-dimensional vertical plane. Let x denote the horizontal position and y the vertical position measured from the ground.

The velocity and acceleration of the ball are

$$\mathbf{v} = \begin{pmatrix} v_x \\ v_y \end{pmatrix}, \qquad \mathbf{a} = \begin{pmatrix} a_x \\ a_y \end{pmatrix}.$$

The gravitational acceleration is constant and directed downward.

Unless explicitly stated otherwise, the ball is treated as a point mass for its translational motion, and effects such as wind and lift are ignored.

Use

$$m = 0.20 \text{ kg}, \qquad A = 3.0 \times 10^{-3} \text{ m}^2, \qquad C_d = 0.50,$$

$$\rho_0 = 1.225 \text{ kg m}^{-3}, \qquad H = 8500 \text{ m}, \qquad g = 9.81 \text{ m s}^{-2}.$$

The initial height and launch speed are

$$h_0 = 1.0 \text{ m}, \qquad v_0 = 25.0 \text{ m s}^{-1}.$$

Atmospheric model

Omar models the density of the atmosphere as a function of altitude according to

$$\rho(y) = \rho_0 e^{-y/H}.$$

Thus, the density used in the drag calculation must be updated according to the ball's instantaneous altitude during the simulation.

The aerodynamic drag force is

$$\mathbf{F}_d = -\frac{1}{2} C_d A \rho(y) |\mathbf{v}| \mathbf{v}.$$

Therefore, the acceleration of the ball is determined by both gravity and the altitude-dependent aerodynamic drag.

Subtask 1 (4 pts): Omar's first launch

Omar begins with a launch angle of

$$\alpha = 45^\circ.$$

The initial conditions are therefore

$$x(0) = 0, \qquad y(0) = h_0,$$

$$v_x(0) = v_0 \cos\alpha, \qquad v_y(0) = v_0 \sin\alpha.$$

Using the atmospheric model given above, simulate the ball's trajectory until it first reaches the ground.

The ball is considered to have reached the ground when

$$y \leq 0.$$

For the purpose of determining the landing position, use the position corresponding to the first timestep at which this condition is satisfied.

Output:

(a) Plot the trajectory $y(x)$ of the ball.

(b) Plot $v_x(t)$ and $v_y(t)$ against time on the same graph.

(c) Determine the maximum height reached by the ball.

(d) Determine the horizontal distance traveled from the launch point until the first impact with the ground.

Subtask 2 (5 pts): Omar changes the atmosphere

Omar now wants to determine whether the variation of atmospheric density has a significant effect on the trajectory.

He performs two simulations using exactly the same initial conditions and physical parameters as in Subtask 1.

In the first simulation, use the realistic altitude-dependent atmosphere

$$\rho(y) = \rho_0 e^{-y/H}.$$

In the second simulation, assume that the atmosphere has a constant density equal to its ground-level value,

$$\rho(y) = \rho_0.$$

All other aspects of the model must remain unchanged.

For both simulations, continue until the ball first reaches the ground.

Output:

(a) Plot the two trajectories on the same x–y graph.

(b) Plot the horizontal velocity v_x against time for both atmospheric models.

(c) Report the maximum height reached in each simulation.

(d) Report the horizontal distance traveled before the first impact in each simulation.

(e) Calculate the percentage difference between the two horizontal distances.

Subtask 3 (5 pts): Omar adds a bouncing surface

The team now places a horizontal surface at

$$y = 0.$$

Omar wants to investigate what happens when the ball is allowed to bounce instead of stopping at its first impact.

The collision with the ground is modeled using a coefficient of restitution

$$e = 0.75.$$

Let v_y^- denote the vertical velocity immediately before an impact and v_y^+ the vertical velocity immediately after the impact.

At each collision, the vertical component of velocity is updated according to

$$v_y^+ = -e\, v_y^-.$$

The horizontal velocity is unchanged during the collision:

$$v_x^+ = v_x^-.$$

No horizontal impulse is produced by the ground, and the collision itself is assumed to occur instantaneously.

After each bounce, the ball continues its motion under gravity and altitude-dependent air resistance.

Omar launches the ball using

$$\alpha = 45^\circ$$

with the same initial speed and height as before.

Simulate the complete motion until the ball has completed exactly **three bounces**.

A bounce is counted whenever the ball reaches the ground while moving downward. The simulation must distinguish an impact from the subsequent motion away from the ground.

Output:

(a) Plot the complete trajectory $y(x)$ from launch until the third bounce.

(b) Clearly mark the three impact points on the trajectory.

(c) Report the horizontal position x_1, x_2, and x_3 of the first, second, and third bounces.

(d) Report the total horizontal distance from the launch point to the third bounce.

(e) Plot the vertical velocity immediately before and immediately after each bounce.

Subtask 4 (6 pts): Omar searches for the best launch angle

Omar's team is now interested in optimizing the launcher.

The initial speed remains fixed at

$$v_0 = 25.0 \text{ m s}^{-1},$$

and the initial height remains

$$h_0 = 1.0 \text{ m}.$$

Only the launch angle α may be changed.

For every launch angle in the range

$$5^\circ \leq \alpha \leq 85^\circ,$$

with an angular resolution of

$$0.1^\circ,$$

Omar must simulate the ball until exactly three bounces have occurred.

For each launch angle, define the total horizontal distance as the horizontal position of the third bounce,

$$D(\alpha) = x_3.$$

The atmospheric model, drag parameters, coefficient of restitution, gravitational acceleration, timestep, and numerical integration method must all remain identical to those used in the previous subtasks.

Omar does not know in advance which angle will produce the greatest distance.

Output:

(a) Calculate $D(\alpha)$ for every launch angle in the specified range.

(b) Plot the total horizontal distance after three bounces, $D(\alpha)$, as a function of the launch angle.

(c) Determine the launch angle α_{opt} that produces the maximum value of $D(\alpha)$.

(d) Report the corresponding maximum distance $D(\alpha_{\text{opt}})$.

(e) Plot the trajectory corresponding to the optimal launch angle and clearly mark its three bounce points.

The optimal angle must be obtained from the numerical search. Do not assume that the optimal angle is 45°.

Problem 1.5

Dr. Farida's Ferromagnetic Film

Dr. Farida, a condensed-matter physicist, studies a thin ferromagnetic film. Below its Curie temperature T_c, neighboring atomic spins tend to align and give the material a net magnetization. Above T_c, thermal motion overwhelms this alignment and the magnetization collapses toward zero.

Model the film as a two-dimensional $L \times L$ lattice of spins $s_i \in \{-1, +1\}$. Each spin

interacts with its four nearest neighbors, with periodic boundary conditions. The energy and total magnetization are

$$E = -J \sum_{\langle i,j \rangle} s_i s_j - h \sum_i s_i, \qquad M = \sum_i s_i.$$

Use single-spin Metropolis updates: accept every flip that lowers the energy; otherwise accept it with the appropriate Boltzmann probability. The magnetic susceptibility is

$$\chi = \frac{\langle M^2 \rangle - \langle M \rangle^2}{k_b T}.$$

Use $J = 1$ and $k_b = 1$ throughout.

Subtask 1 (4 pts): Watching the film settle

Simulate an $L = 40$ film at $T = 1$ with no external field for 400 full lattice sweeps.

Output: Display the final spin arrangement as an image, and plot the total energy and total magnetization over the simulation.

Subtask 2 (5 pts): A sharper estimate from the collapse point

Using $L = 20$ and a small applied field, scan temperatures from $T = 0.5$ to $T = 5.0$ in steps of 0.25, recording the final normalized magnetization. Use the two temperatures straddling the magnetization drop to interpolate a more precise crossing point.

Output: Print the refined estimate of T_c. Plot normalized magnetization against temperature and clearly mark the interpolated crossing point.

Subtask 3 (5 pts): Finding the transition through fluctuations

With no external field, repeat the temperature scan after discarding an initial equilibration period at each temperature. Compute χ from the fluctuation formula.

Output: Plot susceptibility against temperature. Find and print the peak temperature automatically, and compare it with the exact value

$$T_c = \frac{2}{\ln(1 + \sqrt{2})} \approx 2.269.$$

Subtask 4 (6 pts): Does sample size matter?

Repeat the susceptibility scan for $L = 10$, 20, and 30.

Output: Overlay the three susceptibility curves on one plot with a legend. Print a table giving each lattice size, its peak-susceptibility temperature, and its deviation from the exact value. In one printed sentence, describe how increasing L affects the peak's sharpness and proximity to the true transition temperature.

2. Computational Biology and Neuroscience

Problem 2.1

Local Alignment and Motif Discovery

A researcher suspects a conserved motif is shared between two noisy sequences:

Sequence 1: `GATTACAGGCTAGCATTGAC`
Sequence 2: `CCGATTACAGCTTAGCCT`

Use the scoring scheme

$$\text{match} = +3, \qquad \text{mismatch} = -2, \qquad \text{gap} = -3.$$

Subtask 1 (9 pts)

Implement Smith–Waterman from scratch. Report the maximum score, its matrix position, and the traceback alignment, including the aligned substrings, matches, mismatches, gaps, and percentage identity.

Subtask 2 (6 pts)

Re-run the algorithm with a gap penalty of -5 instead of -3. Report whether the maximum score, the alignment length, or the aligned motif itself changes.

Subtask 3 (5 pts)

The researcher expected the conserved motif to be roughly 12–15 bp. Using your two results, state whether the data support this and briefly explain, in one or two sentences, why a stricter gap penalty pushed the result in that direction (or did not).

Problem 2.2

Gene Finding with a Hidden Markov Model

Consider a two-state hidden Markov model with states C (coding) and N (non-coding).

The transition probabilities are

$$P(C \mid C) = 0.85, \qquad P(N \mid C) = 0.15,$$

$$P(C \mid N) = 0.20, \qquad P(N \mid N) = 0.80.$$

The start probabilities are

$$P(C) = 0.6, \qquad P(N) = 0.4.$$

The emission probabilities are

State	A	T	G	C
C	0.15	0.15	0.35	0.35
N	0.35	0.35	0.15	0.15

The observed sequence is

`GCGCATATGGCGATAT`.

Subtask 1 (8 pts)

Implement the Viterbi algorithm from scratch in log-space. Report the most likely state path and its log-probability.

Subtask 2 (5 pts)

Change the nucleotide at position 7 from A to G and re-run Viterbi. Report whether the state at position 7 changes, whether any other positions flip, and the new log-probability.

Subtask 3 (7 pts)

In 1–2 sentences, explain why changing one emitted symbol can shift the predicted state. Then, using your Viterbi matrix from Subtask 1, find the single best alternative path that disagrees with the optimal path at exactly one position, and report the log-probability gap between the two. A small gap means low confidence; a large gap means high confidence.

Problem 2.3

LIF Neuron: Build, Match, Measure

Subtask 1—Baseline model (6 pts)

Implement a leaky integrate-and-fire neuron from scratch using Euler integration:

$$\tau \frac{dV}{dt} = -(V - V_{\text{rest}}) + RI.$$

Use any reasonable starting parameters $(\tau, V_{\text{rest}}, V_{\text{th}}, V_{\text{reset}}, R, I)$. Simulate 200 ms with $\Delta t = 0.1$ ms. Plot $V(t)$ and mark the spikes.

Subtask 2—Match the target trace (8 pts)

A target neuron's voltage trace over 200 ms was produced by a regularly firing LIF neuron with a fixed, undisclosed parameter set. By adjusting your model's parameters $(\tau, V_{\text{rest}}, V_{\text{th}}, V_{\text{reset}}, R, I)$, with Δt fixed at 0.1 ms, find a parameter set that reproduces the following measured properties of the target trace, each within the stated tolerance:

- total spike count over 200 ms: 26 ± 1;
- time of first spike: $7.8\,\text{ms} \pm 1.0\,\text{ms}$;
- mean inter-spike interval: $7.4\,\text{ms} \pm 0.5\,\text{ms}$.

Report your final parameter set and the resulting spike count, first-spike time, and mean inter-spike interval, confirming that they fall within tolerance. Briefly note which parameter or parameters had the largest effect on matching the target.

Subtask 3—Spike detection and firing rate (6 pts)

Write a spike-detection function that scans your matched $V(t)$ trace and records spike times. Do not simply reuse the times from your simulation loop; detect them from the trace itself, for example by finding threshold crossings. Report the total spike count, mean inter-spike interval, and firing rate in Hz.

Problem 2.4

Coalescent Simulation

Consider a sample of $n = 6$ lineages evolving under the Kingman coalescent. When k lineages remain, the waiting time until the next coalescent event is

$$T_k \sim \text{Exponential}(\lambda_k), \qquad \lambda_k = \binom{k}{2} = \frac{k(k-1)}{2}.$$

Each coalescent event merges two lineages chosen uniformly at random from the k currently present.

Subtask 1 (7 pts)

Using `random.seed(42)`, simulate the coalescent process from T_6 down to T_2. Report each waiting time T_k, the total tree height ($\sum_k T_k$), and the total branch length ($\sum_k k \cdot T_k$).

Subtask 2 (7 pts)

Track the merge history of the simulation and output the resulting genealogy as a tree in Newick format, including branch lengths.

Subtask 3 (6 pts)

Compute the theoretical expectation $E[T_k] = 1/\lambda_k$ for each k, and hence the expected total tree height. State whether your simulated height from Subtask 1 came in above or below this expectation (a single comparison is sufficient; a full sampling distribution is not required).

Then derive, from first principles, why the coalescence rate is $\lambda_k = \binom{k}{2}$: note that any one specific pair of lineages coalesces at rate 1, that there are $\binom{k}{2}$ equally likely pairs, and that the time to the first of $\binom{k}{2}$ independent exponential races has a rate equal to the sum of the individual rates. State the probability that any one specific pair is the pair that coalesces at the next event.

Problem 2.5

Calcium Imaging: From Spikes to Fluorescence

Calcium imaging does not record voltage directly. It records a fluorescence signal that rises sharply after a spike and decays slowly, approximately following

$$\frac{\Delta F}{F}(t) = \sum_{\text{spikes}} A \exp\left(-\frac{t - t_{\text{spike}}}{\tau_{\text{decay}}}\right), \qquad t \geq t_{\text{spike}}.$$

Subtask 1—Generate a calcium trace from spikes (6 pts)

Using the spike times you detected in the LIF Neuron problem, Subtask 3, generate a synthetic calcium fluorescence trace $\Delta F/F(t)$ using $A = 1.0$ and $\tau_{\text{decay}} = 400\,\text{ms}$, sampled at the same timestep. Add Gaussian noise with $\sigma = 0.05$ to the trace and plot it.

Subtask 2—Recover spikes from the calcium trace (8 pts)

Calcium traces blur together spikes that are close in time, so simple thresholding of the raw signal will not recover individual spikes. Implement a method to infer spike times from the noisy calcium trace, for example a derivative-based approach that looks for sharp rises rather than absolute fluorescence level. Report the inferred spike count and inferred firing rate, and briefly describe your method.

Subtask 3—Compare with ground truth (6 pts)

Compare your inferred spike count and timing from Subtask 2 with the true spike times used to generate the trace in Subtask 1. Report how many spikes were correctly detected, how many were missed, and how many were falsely detected. In one or two sentences, explain why calcium imaging tends to underestimate firing rate compared with direct electrical recording, especially at high firing rates.

3. Computational Chemistry

Problem 3.1

Chemical reactions can be modeled as systems of linear equations where mass conservation acts as a strict mathematical constraint, allowing for the solution of linear systems for reaction balancing.

The law of conservation of mass dictates that the net change for each element in a closed system must be zero. This can be represented mathematically as $Ax = 0$, where A is the atomic composition matrix, x is the vector of stoichiometric coefficients, and 0 is a zero matrix with n rows and m columns, where n is the number of rows of the atomic composition matrix, and m is the number of columns of the stoichiometric coefficients matrix.

Consider the unbalanced complete combustion of glucose: $a\mathrm{C_6H_{12}O_6} + b\mathrm{O_2} \rightarrow c\mathrm{CO_2} + d\mathrm{H_2O}$.

Subtasks:

1. **(6 pts)** Construct the atomic matrix representing the elemental composition of the reactants and products for the given reaction.
2. **(7 pts)** Write a Python script using `sympy` to compute the null space of the atomic matrix to derive the exact stoichiometric coefficients.
3. **(7 pts)** Normalize the resulting vector so that all stoichiometric coefficients are the smallest possible integers and print this vector, and write a condition to programmatically verify the atom conservation constraints for Carbon, Hydrogen, and Oxygen according to the equation of $Ax = 0$.

Problem 3.2

Molecular descriptors map chemical structures to numerical feature spaces, enabling the rapid prediction of physicochemical properties like bioavailability. A common application is evaluating candidates against Lipinski's Rule of Five for drug-likeness.

The extraction of physicochemical and topological features includes evaluating the partition coefficient (LogP), molecular mass, and hydrogen bond donors and acceptors.

Use the following SMILES strings representing five distinct compounds:

- Aspirin: `CC(=O)OC1=CC=CC=C1C(=O)O`
- Caffeine: `CN1C=NC2=C1C(=O)N(C(=O)N2C)C`
- Testosterone: `CC12CCC3C(C1CCC2O)CCC4=CC(=O)CCC34C`
- Phenylalanine: `C1=CC=C(C=C1)CC(C(=O)O)N`
- Thyroxine: `C1=C(C=C(C(=C1I)O)I)C2=CC(=C(C(=C2)I)O)I`

Subtasks:

1. **(6 pts)** Parse the SMILES strings into molecular object structures using `rdkit`. Compute the molecular mass and partition coefficient (LogP) for each.
2. **(7 pts)** Compute the hydrogen bond donor and acceptor counts for each molecule. Create a bar graph that shows the number of hydrogen bond donors for each molecule and another bar graph that shows the number of hydrogen bond acceptors for each molecule.
3. **(7 pts)** Print the molecular weight, LogP value, Hydrogen bond donors, and Hydrogen bond acceptors for each molecule. Then, conceptually derive and print which of these molecules violates Lipinski's rules based on your computed dataset and explain the physical meaning of a high LogP value in a biological system.

Problem 3.3

Quantifying the similarity between chemical structures is vital for drug discovery and dataset aggregation, often achieved by generating molecular fingerprints as bit-vector representations.

Algorithms like the Morgan fingerprint encode the topological environment of atoms into a bit-array. Tanimoto Coefficient is a standard metric for computing similarity in chemical space, defined for bit-vectors as $T_c = \frac{c}{a+b-c}$, where c is the number of shared bits, and a and b are the total bits set in each respective vector.

Consider three neurotransmitters:

- Serotonin: `C1=CC2=C(C=C1O)C(=CN2)CCN`
- Dopamine: `C1=CC(=C(C=C1CCN)O)O`
- Epinephrine: `CNCCC(C1=CC(=C(C=C1)O)O)O`

Subtasks:

1. **(6 pts)** Write a Python script to generate and store Morgan fingerprints (radius 2, 2048 bits) for the three neurotransmitters.
2. **(7 pts)** Compute the Tanimoto similarity metrics in chemical space for all possible pairs of these molecules and store them in a 2d matrix with 3 rows and columns, where each row and column represent a molecule, and each cell represents the similarity score between two molecules. Then, visualize the results using a heatmap with (`cmap="YlGnBu", vmin=0, vmax=1`).
3. **(7 pts)** Identify and print the ranking of all the possible pairs starting from the highest to the lowest similarity score. (for example: "[rank] - [molecule1 name] and [molecule2 name] have a similarity score of [X.XX]")

Problem 3.4

Galvanic cells harness spontaneous redox reactions to generate electrical energy, with cell potentials dynamically changing as ionic concentrations shift during operation. Nernst Equation relates dynamic cell potential to the standard potential and the reaction quotient: $E = E^\circ - \frac{RT}{nF} \ln Q$.

Consider a Daniell cell: $\mathrm{Zn}_{(s)} + \mathrm{Cu}^{2+}_{(aq)} \rightleftharpoons \mathrm{Zn}^{2+}_{(aq)} + \mathrm{Cu}_{(s)}$. Use $E^\circ_{cell} = 1.10\,\mathrm{V}$, $T = 298\,\mathrm{K}$, $F = 96485\,\mathrm{C\ mol^{-1}}$, $R = 8.314\,\mathrm{J\ mol^{-1}K^{-1}}$. Initial conditions (1.0 L volume): $[\mathrm{Cu}^{2+}] = 1.0\,\mathrm{M}$, $[\mathrm{Zn}^{2+}] = 0.01\,\mathrm{M}$.

Subtasks:

1. **(6 pts)** Implement an application of the Nernst equation in computational settings to calculate the reaction quotient Q and the cell potential E as the concentration of Cu^{2+} depletes from 1.0 M to 0.01 M in 0.01 M decrements and Zn^{2+} is increasing at the same rate.
2. **(7 pts)** Plot the calculated cell potential E (y-axis) versus $\ln Q$ (x-axis). Perform a linear regression on the plotted data to extract and print the slope, and conceptually derive and print the theoretical constant slope $-\frac{RT}{nF}$.
3. **(7 pts)** Based on an application of Faraday's laws in computational settings $Q = nF$, where Q is the total electric charge in coulombs (C), n is the number of moles of electrons transferred (mol), and F is Faraday's Constant, calculate and print the total charge (in Coulombs) transferred when the cell potential reaches exactly 1.05 V.

Problem 3.5

Phase transitions and chemical reactions require an accounting of energy flows. These physical changes are rigorously described through computational thermochemistry.

The energy required to change temperature and phase is dictated by the Laws of Thermodynamics. Enthalpy (ΔH) calculations depend on temperature-dependent heat capacities, $C_p(T) = a + bT + cT^2$.

Consider heating 1 mole of $\mathrm{H_2O}$ from solid (250 K) to gas (500 K) at 1 atm.

- $C_p(\mathrm{ice}) = 38.0\,\mathrm{J\ mol^{-1}K^{-1}}$
- $C_p(\mathrm{liq}) = 75.3\,\mathrm{J\ mol^{-1}K^{-1}}$
- $C_p\ (\mathrm{steam}) = 30.0T + 5.35 \times 10^{-3}T^2 + 0.11 \times 10^{-5}T^3$ $\mathrm{J\ mol^{-1}K^{-1}}$ (Integrated

form, ready to be used)

- $\Delta H_{\text{fusion}} = 6.01\,\text{kJ mol}^{-1}$
- $\Delta H_{\text{vap}} = 40.65\,\text{kJ mol}^{-1}$

Consider Syngas Reaction: $\text{C}_{(s)} + \text{H}_2\text{O}_{(g)} \rightleftharpoons \text{CO}_{(g)} + \text{H}_{2(g)}$

- $\Delta H^\circ_{rxn} = 131.3\,\text{kJ mol}^{-1}$
- $\Delta S^\circ_{rxn} = 133.6\,\text{J mol}^{-1}\text{K}^{-1}$

Subtasks:

1. **(6 pts)** Implement a script using numerical integration to calculate and print the total enthalpy change (ΔH) in KJ/mol for heating the water from 250 K to 500 K, executing the necessary Heat capacity, specific heat capacity, and latent heat calculations.
2. **(7 pts)** Applying the Law of change in Internal energy, compute and print the internal energy change ($\Delta U = \Delta H - P\Delta V$) for this entire heating process in KJ/mol. Assume ideal gas behavior for steam ($PV = nRT$) and negligible volume for ice and liquid water.
3. **(7 pts)** For the syngas reaction, write a function to perform Gibbs free energy calculations over the range of 400 K to 1500 K. Based on Thermodynamic stability criteria, computationally find the exact inversion temperature in K where the reaction shifts to being spontaneous.

4. Computational Mathematics

Problem 4.1

Simpson's Aqueduct

Heron of Alexandria (c. 10–70 AD) was an engineer and mathematician who designed surveying instruments, early steam-powered devices, and wrote extensively on mensuration. Roman aqueduct engineers, inheriting this tradition, needed to estimate the cross-sectional flow area of channels whose walls were gentle curves, measured only at intervals.

An aqueduct engineer has surveyed the depth profile of a channel and modelled its cross-sectional shape by the function:

$$y(x) = 4 - \frac{(x-2)^2}{3}, \qquad \text{for } 0 \le x \le 4 \qquad (x, y \text{ in metres})$$

The flow area is the definite integral $\int_0^4 y(x)\,dx$. The engineer wants two independent numerical estimates of this area before trusting either, plus a check against the exact symbolic answer.

Subtask 1: Trapezoidal Rule (4 pts)

Write a program that implements the Trapezoidal Rule from scratch to estimate $\int_0^4 y(x)\,dx$ using $n = 8$ equal subintervals. Report the estimate.

Subtask 2: Simpson's Rule (6 pts)

Using $n = 8$ equal subintervals, implement Simpson's Rule from scratch and report the estimate for the same integral. Plot your numerical approximations alongside the exact function $y(x)$, displaying the absolute error of each method in the legend.

Subtask 3: Symbolic Ground Truth (4 pts)

Use SymPy to compute the exact value of $\int_0^4 y(x)\,dx$ symbolically. Report the absolute error of your Trapezoidal and Simpson estimates against this exact value.

Subtask 4: Convergence Plot (6 pts)

Using Matplotlib, plot the absolute error of the Trapezoidal Rule and Simpson's Rule against n, for $n \in \{2, 4, 8, 16, 32, 64\}$ using a logarithmic scale on the error axis.

Problem 4.2

Newton's Impatience

Sir Isaac Newton (1642–1727) developed his method for finding roots while working on his treatise *Method of Fluxions*. Newton was famously impatient with slow-converging techniques; where Bisection Method halves the search space at each step, Newton's method uses local slopes to leap towards the root.

Newton wants to find the root of $f(x) = e^x - 3x^2$ near $x = 4$, but insists your program should not require an explicit user-provided derivative formula.

Subtask 1: Numerical Derivative (5 pts)

Write a function `central_diff(f, x, h)` that returns the central finite-difference approximation of $f'(x)$ using step size h. Report $f'(4)$ using $h = 10^{-5}$, and compute the absolute error against the true analytical derivative calculated using SymPy.

Subtask 2: Newton–Raphson from Scratch (6 pts)

Using `central_diff` in place of an exact derivative, implement Newton–Raphson to find the root of $f(x)$ starting from $x_0 = 4$. Iterate until $|f(x_n)| < 10^{-8}$ or 100 iterations are reached. Report the computed root and the iteration count.

Subtask 3: Programmatic Failure Detection (9 pts)

Modify your Newton–Raphson solver to include guard logic for small derivatives: if $|f'(x_n)| < 10^{-6}$, return a failure flag `"SMALL_DERIVATIVE_WARNING"`. Run this updated solver on $f(x) = e^x - 3x^2$ starting from $x_0 = 0$.

Write code that records and reports:

a) The initial slope $f'(x_0)$ and the magnitude of the initial step $|\Delta x_0| = \left|\dfrac{f(x_0)}{f'(x_0)}\right|$.

b) The iteration index at which the numerical failure guard triggers.

Problem 4.3

The Astronomer's Bracket

Hypatia of Alexandria (c. 350–415 AD) was a mathematician and astronomer who led the Neoplatonic school in Alexandria. While calibrating a new astrolabe, Hypatia models the apparent altitude error of a star, as a function of dial angle x (in radians), by $g(x) = x^3 - 2x - 5$.

Subtask 1: Bracketing the Root (5 pts)

Write a program that evaluates $g(x)$ at integer points $x \in \{0, 1, 2, 3\}$ and programmatically detects the smallest integer-endpoint bracket $[a, b]$ where $g(a) \cdot g(b) < 0$. Report the bracket $[a, b]$.

Subtask 2: Bisection (8 pts)

Using $[a, b]$ from Subtask 1, implement the Bisection Method from scratch to find the root of $g(x)$ to a tolerance of $|b - a| < 10^{-6}$. Report the root and the required iteration count N_{actual}.

Subtask 3: Analytical vs Practical Iterations (7 pts)

Write a code snippet that analytically calculates the theoretical minimum iteration count

$$N_{\text{theory}} = \left\lceil \log_2 \left(\frac{b - a}{\varepsilon} \right) \right\rceil$$

using $\varepsilon = 10^{-6}$. Report N_{theory} and programmatically assert that $N_{\text{actual}} == N_{\text{theory}}$.

Problem 4.4

Al-Biruni's Survey

Abu Rayhan al-Biruni (973–1048) was a polymath who made major contributions to geodesy and survey mathematics. An apprentice has recorded noisy elevation measurements along a candidate canal route:

Distance x (km)	0	1	2	3	4	5
Elevation y (m)	12.1	14.8	17.3	20.4	22.9	25.5

Subtask 1: Line of Best Fit (5 pts)

Write a program that computes the best-fit line $y = mx + c$ for this dataset. You can solve this by constructing the standard matrix equation $X\beta = y$ and solving $X^T X\beta = X^T y$ (for instance, using `numpy.linalg.solve` or `numpy.polyfit`).

Plot the actual data points alongside your fitted line, and report the slope m and intercept c.

Subtask 2: Point Predictions (4 pts)

Estimate the elevation at $x = 2.5$ km using two different methods:

a) y_{interp}: Straight-line interpolation halfway between the two nearest points, $(2, 17.3)$ and $(3, 20.4)$.

b) y_{fit}: Your linear equation $y = mx + c$ from Subtask 1 evaluated at $x = 2.5$.

Report y_{interp}, y_{fit}, and the absolute difference $|y_{\text{interp}} - y_{\text{fit}}|$.

Subtask 3: Measuring Error (Sum of Squared Errors) (5 pts)

To compare how well both methods fit the data, write a program to calculate the Sum of Squared Errors ($\text{SSE} = \sum(y_{\text{actual}} - y_{\text{predicted}})^2$) for both the linear interpolation method and the best-fit line:

a) **Total SSE:** The sum of squared errors across all 6 data points ($x = 0, 1, 2, 3, 4, 5$).

b) **Local SSE:** The sum of squared errors evaluated only at $x = 2$ and $x = 3$.

Report both metrics ($\text{SSE}_{\text{total}}$ and $\text{SSE}_{\text{local}}$) for both methods.

Subtask 4: Far-Out Prediction Risk (Leverage) (6 pts)

Predicting values far outside your data range (extrapolation) carries higher risk.

a) Use your best-fit line equation to estimate the elevation at $x = 20$ km.

b) Compute the leverage score h_{20}, which measures how far $x = 20$ is from the center of your data:

$$h_{20} = \frac{1}{n} + \frac{(20 - \bar{x})^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

(Alternatively, using matrix form: $h_{20} = x_{20}^T (X^T X)^{-1} x_{20}$ where $x_{20} = [1, 20]^T$.)

Report predicted elevation $y(20)$ and the leverage score h_{20}.

Problem 4.5

Halley's Comet

A falling body subject to gravity and air resistance obeys the differential equation:

$$\frac{dv}{dt} = -g - kv, \qquad v(0) = 0$$

where $g = 9.8$ m/s^2 and $k = 0.2$ s^{-1}. The exact analytical velocity profile is $v(t) = -\frac{g}{k}\left(1 - e^{-kt}\right)$.

Subtask 1: Euler's Method from Scratch (4 pts)

Implement Euler's Method for $t \in [0, 20]$ with $h = 0.5$ s. Report $v(20)$ and its absolute error relative to $v_{\text{exact}}(20)$.

Subtask 2: RK4 from Scratch (5 pts)

Implement the classical 4th-Order Runge–Kutta method (RK4) using $h = 0.5$ s. Report $v(20)$ and its absolute error relative to $v_{\text{exact}}(20)$.

Subtask 3: Programmatic Convergence Order (7 pts)

For step sizes $h \in \{2, 1, 0.5, 0.25, 0.125, 0.0625\}$, compute terminal error $E(h) = |v_{\text{num}}(20) - v_{\text{exact}}(20)|$ for both Euler and RK4. Plot $\log_{10}(E(h))$ vs $\log_{10}(h)$. Fit a straight line $\log_{10}(E(h)) = p \log_{10}(h) + C$ using linear regression for both methods, and report the extracted numerical slopes p_{Euler} and p_{RK4}.

Subtask 4: Cross-Check with SciPy (4 pts)

Solve the ODE using `scipy.integrate.solve_ivp` (with `atol=1e-10, rtol=1e-10`). Report the absolute difference between the RK4 solution at $h = 0.5$ s and the `solve_ivp` numerical solution at $t = 20$.